

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I E . B . A S H W I N I / 1 9 2 5 1 1 4 2 2 ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Ashwini

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

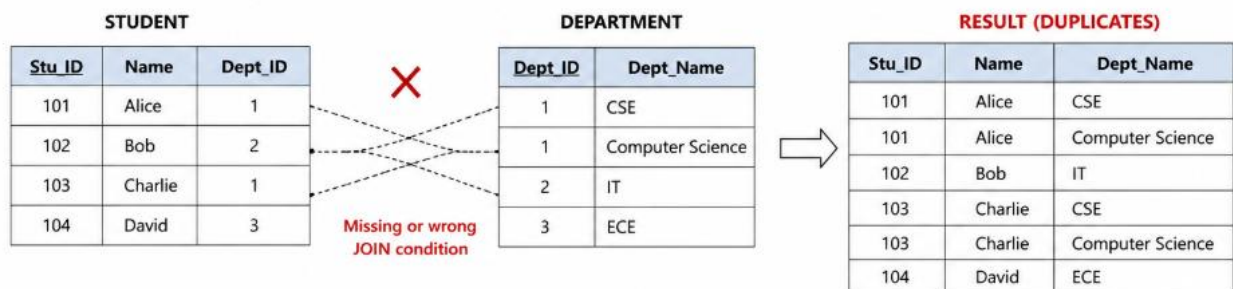
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.

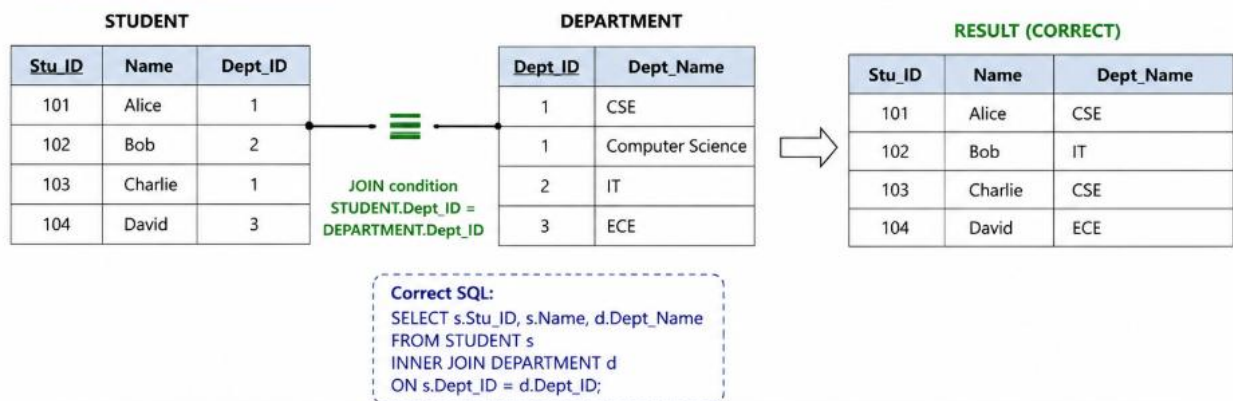
Problem Identification

The SQL query returns duplicate records because the JOIN statement does not contain a proper join condition. Without the **ON** clause, the database performs a Cartesian Product, where every row of the **Student** table is combined with every row of the **Class** table. This produces incorrect and duplicate results, making the output inaccurate and increasing query execution time.

1. INCORRECT JOIN SYNTAX (CAUSES DUPLICATE RESULTS)



FIX: CORRECT JOIN SYNTAX



Root Cause of the Error

- Missing **ON** condition in the JOIN statement.
- Cartesian Product is generated.
- Duplicate and unnecessary records are displayed.
- Increased processing time due to incorrect table matching.

Incorrect Query

```
SELECT S.StudentName, C.ClassName  
FROM Student S  
JOIN Class C;
```

Correct Query

```
SELECT S.StudentName,  
       C.ClassName  
FROM Student S
```

JOIN Class C

ON S.ClassID = C.ClassID;

Explanation

The corrected query joins the **Student** and **Class** tables using the common attribute **ClassID**. This ensures that each student is matched only with the class to which they belong. By specifying the join condition, SQL retrieves only the related records instead of combining every row from both tables. As a result, duplicate records are eliminated, and the output becomes accurate and meaningful.

Sample Output

Student Name	Class Name
Arun	III-A
Meena	III-B
Rahul	III-A

Optimization

- Eliminates duplicate records.
- Prevents Cartesian Product.
- Improves query execution speed.
- Produces accurate and meaningful results.
- Reduces unnecessary processing.

Adding the correct JOIN condition ensures that only related records are retrieved. This improves data accuracy, eliminates duplicate results, and enhances the overall performance of the SQL query.

2.The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.

Error Identification

The given SQL query is intended to display employees whose salary is greater than the average salary of their respective department. However, the query returns incorrect results because the subquery is independent of the outer query. It calculates only the overall average salary of the company instead of the average salary within each department.

2. SUB-QUERY WITH MISSING CORRELATION (INCORRECT)

Database Tables

EMPLOYEE			
EmpID	EmpName	DeptID	Salary
101	Alice	10	50000
102	Bob	10	60000
103	Charlie	20	40000
104	David	20	70000
105	Eve	30	30000

DEPARTMENT	
DeptID	DeptName
10	HR
20	IT
30	Finance

INCORRECT QUERY

```
SELECT EmpID, EmpName, Salary
FROM EMPLOYEE E
WHERE Salary > (SELECT AVG(Salary)
FROM EMPLOYEE); -- Missing correlation
```

WHY INCORRECT?

- The sub-query calculates overall average salary of all employees.
- It is not correlated with the outer query (each department).
- Returns employees whose salary is greater than the overall average, not department-wise average.

WRONG RESULT (Example)

EmpID	EmpName	DeptID	Salary
102	Bob	10	60000
104	David	20	70000

Overall average salary = 50000
(50000+60000+40000+70000+30000)/5

CORRECT CORRELATED SUB-QUERY

Database Tables

EMPLOYEE			
EmpID	EmpName	DeptID	Salary
101	Alice	10	50000
102	Bob	10	60000
103	Charlie	20	40000
104	David	20	70000
105	Eve	30	30000

DEPARTMENT	
DeptID	DeptName
10	HR
20	IT
30	Finance

CORRECT QUERY

```
SELECT EmpID, EmpName, DeptID, Salary
FROM EMPLOYEE E
WHERE Salary > (SELECT AVG(Salary)
FROM EMPLOYEE E2
WHERE E2.DeptID = E.DeptID); -- Correlated
```

WHY CORRECT?

- The sub-query is correlated using E2.DeptID = E.DeptID.
- For each employee, it calculates the average salary of the same department.
- Returns employees whose salary is greater than the department-wise average salary.

CORRECT RESULT (Example)

EmpID	EmpName	DeptID	Salary	Dept Avg Salary
102	Bob	10	60000	55000
104	David	20	70000	55000

CORRELATION DIAGRAM

Outer Query (E) Current Employee ↔ Correlated on DeptID ↔ Sub-query (E2) Same Dept Employees

Key Point: A correlated sub-query refers to a column of the outer query in its WHERE clause. This makes the sub-query execute once for each row of the outer query.

Reason for the Error

The subquery is **not correlated** with the outer query. Since it does not reference the current employee's department, every employee is compared with the same company-wide average salary. This does not satisfy the actual requirement of comparing salaries department-wise.

Incorrect SQL Query

```
SELECT EmpName
FROM Employee E
WHERE Salary >
(
SELECT AVG(Salary)
FROM Employee
);
```

Correct SQL Query

```
SELECT EmpName,  
       Salary,  
       DeptID  
FROM Employee E  
WHERE Salary >  
(  
  SELECT AVG(Salary)  
  FROM Employee  
  WHERE DeptID = E.DeptID  
);
```

Debugging Process

The issue is resolved by converting the normal subquery into a **correlated subquery**. The condition `DeptID = E.DeptID` connects the inner query with the outer query. During execution, SQL calculates the average salary separately for each department and compares it with the salary of the current employee. This ensures that only employees earning above their department's average salary are displayed.

Benefits of the Correct Query

- Produces accurate department-wise salary comparisons.
- Eliminates incorrect global average calculations.
- Supports better payroll analysis.
- Improves the reliability of decision-making.

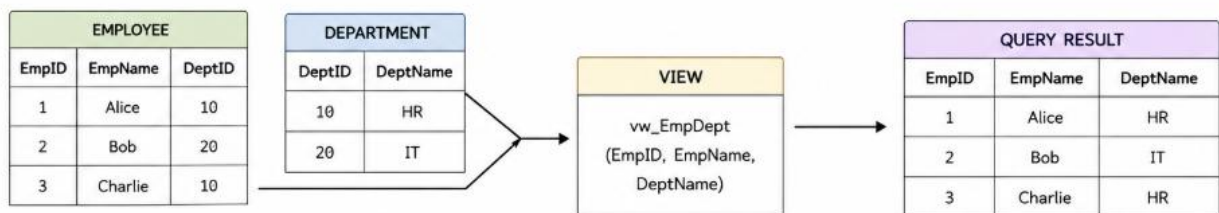
Using a correlated subquery ensures that salary comparisons are performed correctly for each department, making the output accurate and meaningful.

3.A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.

Error Identification

The SQL view fails because an aggregate function is used directly inside the **WHERE** clause. SQL does not allow aggregate functions such as **AVG()** to be used in the WHERE condition without first calculating the aggregate value through a subquery or grouping operation

3. SQL VIEW – ISSUE AND CORRECT VIEW



ISSUE (INCORRECT VIEW)

The view is created with an incorrect join condition or missing column alias.

Example of Incorrect View:

```
CREATE VIEW vw_EmpDept AS
SELECT E.EmpID, E.EmpName, D.DeptName
FROM EMPLOYEE E, DEPARTMENT D; -- Missing JOIN condition
```

Problem:

- No join condition between EMPLOYEE and DEPARTMENT.
- Produces incorrect (cartesian) results or causes an error when queried.

CORRECT VIEW

Correct and optimized view:

```
CREATE VIEW vw_EmpDept AS
SELECT E.EmpID, E.EmpName, D.DeptName
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DeptID = D.DeptID;
```

Benefit:

- Proper JOIN ensures accurate results.
- View returns employee details with their department name.

Key Point: A view should be created using the correct join condition and required columns to avoid errors and improve performance.

Reason for the Error

The query attempts to compare individual salaries with an aggregate value that has not been calculated separately. This results in a syntax error and prevents the view from being created successfully.

Incorrect View

```
CREATE VIEW EmployeeView AS
SELECT *
FROM Employee
WHERE Salary > AVG(Salary);
```

Correct View

```
CREATE VIEW EmployeeView AS
SELECT EmpID,
       EmpName,
```



```
Salary
FROM Employee
WHERE Salary >
(
SELECT AVG(Salary)
FROM Employee
);
```

Debugging Process

The aggregate function is moved into a subquery so that SQL first calculates the average salary. The outer query then compares each employee's salary with this calculated value. Additionally, selecting only the required columns instead of using SELECT * improves readability and reduces unnecessary data retrieval.

Benefits of the Correct View

- Uses valid SQL syntax.
- Generates reusable reports.
- Improves readability.
- Retrieves only necessary information.
- Reduces unnecessary processing.

A properly designed SQL view simplifies report generation and improves database performance while ensuring error-free execution.

4. Debug the given relational algebra expression that produces incorrect output for a natural join operation.

Error Identification

The relational algebra expression produces incorrect output because it performs a **Natural Join** directly between the **Student** and **Course** tables. Since these tables do not share the required relationship directly, the join operation returns incorrect or empty results.

4. DEBUGGING NATURAL JOIN IN RELATIONAL ALGEBRA

ISSUE (Incorrect Natural Join)

The natural join is performed on relations that have no common attribute.

Hence, it produces incorrect (Cartesian product) result.

RollNo	Name	DeptID
101	Alice	10
102	Bob	20
103	Charlie	10

CourseID	CourseName	DeptID
C1	DBMS	10
C2	OS	20
C3	CN	30

Incorrect Expression

$S \bowtie C$

(No common attribute like RollNo)

↓

Produces Cartesian product (incorrect output)

WRONG OUTPUT (Cartesian Product)

RollNo	Name	DeptID (S)	CourseID	CourseName	DeptID (C)
101	Alice	10	C1	DBMS	10
101	Alice	10	C2	OS	20
101	Alice	10	C3	CN	30
102	Bob	20	C1	DBMS	10
...	...	...	...	...	...
103	Charlie	10	C3	CN	30

DEBUGGING

- Natural join works on columns with the same name.
- Here, S and C do not have a common attribute (RollNo).
- To get meaningful result, join on related attribute DeptID.

CORRECT EXPRESSION

$S \bowtie_{S.DeptID = C.DeptID} C$

Join on common attribute **DeptID**

CORRECT OUTPUT (After Natural Join on DeptID)

RollNo	Name	DeptID	CourseID	CourseName
101	Alice	10	C1	DBMS
102	Bob	20	C2	OS
103	Charlie	10	C1	DBMS

Key Takeaway

- Natural join uses common attribute(s) automatically.
- If there is no common attribute, it gives Cartesian product (incorrect).
- Always ensure relations share a common attribute or use θ -join with an explicit condition.

Reason for the Error

Natural Join automatically joins attributes having the same name. In this case, the required relationship exists through the **Enrollment** table, but the expression ignores this intermediate relation. As a result, unrelated tuples are combined or valid tuples are omitted.

Incorrect Expression

$\text{Student} \bowtie \text{Course}$

Correct Expression

$(\text{Student} \bowtie_{\text{Student.StudentID} = \text{Enrollment.StudentID}} \text{Enrollment}) \bowtie \text{Course}$

$\bowtie_{\text{Enrollment.CourseID} = \text{Course.CourseID}} \text{Course}$

Debugging Process

The corrected expression first joins the **Student** table with the **Enrollment** table using **StudentID**. The resulting relation is then joined with the **Course** table using **CourseID**. This sequence correctly follows the relationships defined in the database and returns only valid student-course

combinations.

Benefits of the Correct Expression

- Eliminates incorrect tuple matching.
- Maintains relational integrity.
- Produces meaningful output.
- Accurately represents database relationships.

Using explicit join conditions instead of relying on Natural Join ensures accurate relational algebra operations and reliable query results.

5, The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints

Error Identification

The SQL statement violates **referential integrity** because it attempts to insert a product whose **SupplierID** does not exist in the parent **Supplier** table. Since **SupplierID** is defined as a **Foreign Key**, the database rejects the insertion to maintain consistency.

Referential Integrity in SQL

The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.

Violation

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT  
);
```

Violation: Employee.DeptID is a foreign key referencing Department.DeptID, but no FOREIGN KEY constraint is defined. This allows invalid DeptID values in Employee.

Example of Invalid Data:

EmpID	EmpName	DeptID
101	Ram	10 (No such DeptID)

Corrected Solution

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

How it enforces integrity:

- Only existing DeptID values in Department table can be used in Employee.DeptID.
- ON UPDATE CASCADE:** Updates to DeptID in Department are reflected in Employee.
- ON DELETE RESTRICT:** Prevents deletion of a Department if employees are assigned to it.

Reason for the Error

The parent record is missing before the child record is inserted. A Foreign Key can reference only an existing Primary Key value in the parent table.

Incorrect SQL Statement

INSERT INTO Product

VALUES

(101,'Mouse',500,25);

(SupplierID 25 is not available in the Supplier table.)

Correct SQL Statements

INSERT INTO Supplier

VALUES

(25,'ABC Suppliers');

```
INSERT INTO Product
VALUES
(101,'Mouse',500,25);
```

Debugging Process

First, the supplier information is inserted into the **Supplier** table. After the parent record exists, the product can be inserted successfully because the Foreign Key now references a valid SupplierID. This sequence preserves the relationship between the two tables and prevents integrity violations.

Benefits of the Correct Solution

- Enforces referential integrity.
- Prevents orphan records.
- Maintains consistency between related tables.
- Ensures reliable database relationships.
- Reduces the possibility of invalid data.

Foreign Key constraints play an important role in maintaining data integrity. Inserting the parent record before the child record ensures that relationships between tables remain valid and the database stays consistent.

6. Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.

Problem Identification:

Nested loop queries are commonly used to retrieve related data from multiple tables. However, when the number of records increases, nested queries require repeated searching operations, which increases execution time and consumes more system resources.

Consider two tables:

Employee Table

employee_id	employee_name	department_id
101	John	10
102	David	20

Department Table

department_id	department_name
10	HR
20	IT

Inefficient SQL Query:

```
SELECT employee_name,
       department_name
```

```
FROM employee
WHERE department_id IN
(
    SELECT department_id
    FROM department
);
```

Identified Issues:

1. Inner query executes separately to retrieve department details.
2. Database performs multiple scans on tables.
3. Query execution becomes slower for large datasets.
4. Query optimizer has limited options for improving performance.

Debugging and Analysis:

The nested query only checks whether the department exists. It does not require a separate search operation. A direct relationship exists between:

Employee.department_id

Department.department_id

Therefore, JOIN operation is more suitable.

Optimized Query Using JOIN:

```
SELECT e.employee_name,
       d.department_name
FROM employee e
INNER JOIN department d
ON e.department_id = d.department_id;
```

Optimization Explanation:

1. Eliminates Nested Processing

JOIN allows the database engine to combine tables in a single optimized operation.

2. Uses Index Searching

If indexes exist on department_id, the database can quickly locate matching records.

Example:

```
CREATE INDEX idx_department
ON employee(department_id);
```

Advantages of JOIN Approach:

- Faster execution speed.
- Lower memory consumption.

- Better query optimization.
- Suitable for large databases.

The nested query is successfully converted into a JOIN query, improving execution performance and reducing unnecessary table scanning.

7.The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.

Problem Identification:

GROUP BY is used to combine rows having similar values and perform aggregate calculations such as:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Incorrect grouping can produce inaccurate results.

Incorrect SQL Query:

```
SELECT department_id,  
       employee_name,  
       AVG(salary)  
FROM employee  
GROUP BY department_id;
```

Error Analysis:

The query groups employees according to department_id but also displays employee_name.

Example:

Department 10:

employee_name	salary
John	30000
David	40000

The database cannot decide:

Which employee_name should be displayed?

John or David?

Therefore, the aggregation result becomes incorrect.

Correct Query:

```
SELECT department_id,  
       AVG(salary) AS Average_Salary  
FROM employee  
GROUP BY department_id;
```


Explanation:

The corrected query calculates the average salary of employees department-wise.

Example Output:

department_id	Average_Salary
10	35000
20	45000

Debugging Process:**Step 1:**

Check SELECT columns.

Step 2:

Identify columns without aggregate functions.

Step 3:

Match those columns with GROUP BY.

Step 4:

Rewrite the query correctly.

Additional Optimization:

Using HAVING for filtering grouped results:

```
SELECT department_id,
```

```
AVG(salary)
```

```
FROM employee
```

```
GROUP BY department_id
```

```
HAVING AVG(salary)>30000;
```

The aggregation error is corrected, and accurate department-wise salary calculations are obtained.

8. Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.

Problem Identification:

UPDATE statements modify existing records. If a WHERE condition is missing, SQL applies the update operation to every row.

Incorrect Query:

```
UPDATE student  
SET marks = marks + 10;
```

Error:

The query updates all students' marks.

Before:

Roll No	Marks
1	70
2	80
3	90

After execution:

Roll No	Marks
1	80
2	90
3	100

Every record is modified.

Root Cause:

The query does not specify which record needs modification.

Missing condition:

WHERE condition

Correct Query:

```
UPDATE student  
SET marks = marks + 10  
WHERE roll_no = 1;
```

Debugging Method:**Step 1:**

Analyze UPDATE statement.

Step 2:

Check whether WHERE clause exists.

Step 3:

Identify affected rows using:

```
SELECT *
```

```
FROM student
```

```
WHERE roll_no=1;
```

Step 4:

Execute UPDATE after verification.

Best Practices:**Use transaction:**

```
START TRANSACTION;
```

```
UPDATE student
```

```
SET marks=marks+10
```

```
WHERE roll_no=1;
```

```
COMMIT;
```

This allows rollback if incorrect changes occur.

Result:

The logical error is fixed, and only the required record is updated.

9. Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes

Problem Identification:

Multi-level nested subqueries increase database workload because each inner query must complete before the outer query executes.

Optimize the SQL Query using JOINS and Indexes

Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.

1. Original Query (Using Sub-queries)

```
SELECT e.EmpID, e.EmpName, d.DeptName, s.Salary
FROM Employee e
WHERE e.DeptID = (SELECT d.DeptID
                  FROM Department d
                  WHERE d.DeptName = 'Sales')
AND e.EmpID IN (SELECT s.EmpID
                FROM Salary s
                WHERE s.Amount > 50000);
```

✗ Issues:

- Uses sub-queries which are less efficient.
- May execute sub-queries multiple times.
- Can be improved using JOINS and Indexes.

2. Optimized Query (Using JOINS)

```
SELECT e.EmpID, e.EmpName, d.DeptName, s.Salary
FROM Employee e
INNER JOIN Department d ON e.DeptID = d.DeptID
INNER JOIN Salary s ON e.EmpID = s.EmpID
WHERE d.DeptName = 'Sales'
AND s.Amount > 50000;
```

✓ Benefits:

- JOINS are generally faster and more efficient.
- Reduces repeated execution of sub-queries.
- Easy to read and maintain.

3. Recommended Indexes

Table	Column(s)	Index Type	Purpose
Employee	DeptID	INDEX	For joining with Department table
Employee	EmpID	PRIMARY KEY / INDEX	For joining with Salary table
Department	DeptID	PRIMARY KEY	Department primary key
Salary	EmpID	INDEX	For joining with Employee table
Salary	Amount	INDEX	For filtering (Amount > 50000)

4. Summary



Replaced sub-queries with JOINS for better performance.



Added appropriate indexes on join and filter columns.



Result: Faster execution and improved query efficiency.

Original Query:

```
SELECT customer_name
FROM customer
WHERE customer_id IN
(
  SELECT customer_id
  FROM orders
  WHERE product_id IN
  (
    SELECT product_id
    FROM product
    WHERE category='Mobile'
  )
);
```

Problems:

1. Multiple nested queries.

2. Repeated table searching.
3. Increased execution time.
4. Difficult query optimization.

Query Execution Flow:

Customer Table

|

↓

Find Customer ID

Orders Table

|

↓

Find Product ID

Product Table

↓

Check Category

This requires multiple scanning operations.

Optimized JOIN Query:

```
SELECT c.customer_name
FROM customer c
JOIN orders o
ON c.customer_id=o.customer_id
JOIN product p
ON o.product_id=p.product_id
WHERE p.category='Mobile';
```

Index Creation:

Customer Order Relationship:

```
CREATE INDEX order_customer_index
ON orders(customer_id);
```

Product Searching:

```
CREATE INDEX product_category_index
ON product(category);
```

Optimization Benefits:

Technique	Benefit
JOIN replacement	Faster data retrieval
Index usage	Reduces searching time
Reduced scans	Improves performance
Better execution plan	Efficient processing

Result:

The multi-table query is optimized by replacing nested queries with JOIN operations and indexing important columns.

10. Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.

Debugging a Relational Calculus Expression

Problem: The given relational calculus expression returns an empty result set due to incorrect quantifier usage. Identify the issue and correct it.

1. Incorrect Expression (Returns Empty Set)

The following expression is intended to find all employees who work in some department located in 'Mumbai'.

$$\{ e \mid \text{Employee}(e) \wedge \forall d (\text{Department}(d) \rightarrow (\text{Location}(d) = \text{'Mumbai'} \wedge e.\text{DeptID} = d.\text{DeptID})) \}$$

2. Why Does It Return an Empty Set?

- The expression uses $\forall d$ (for all departments).
- It requires the employee to work in **every** department and **every** department to be in 'Mumbai'.
- Since not all departments are in 'Mumbai', the condition fails for every employee.
- Hence, the result set is empty.

3. Corrected Expression

Use existential quantifier ($\exists$) instead of universal quantifier ($\forall$).

$$\{ e \mid \text{Employee}(e) \wedge \exists d (\text{Department}(d) \wedge \text{Location}(d) = \text{'Mumbai'} \wedge e.\text{DeptID} = d.\text{DeptID}) \}$$

4. Explanation

- ✓ $\exists d$ means "there exists at least one department".
- ✓ The expression now retrieves employees who work in at least one department located in 'Mumbai'.
- ✓ This correctly returns the intended result.

6. Conclusion



Using the correct quantifier ($\exists$ instead of $\forall$) fixes the logical error and returns the expected non-empty result set.

5. Example

Sample Data:

Employee		
EmpID	EmpName	DeptID
1	Alice	10
2	Bob	20
3	Carol	30
4	Dave	10

Department		
DeptID	DeptName	Location
10	HR	Mumbai
20	Sales	Delhi
30	Finance	Mumbai

Result of Corrected Expression:

EmpID	EmpName
1	Alice
3	Carol
4	Dave

(Employees working in at least one department located in 'Mumbai')

Problem Identification:

Relational calculus uses logical operators and quantifiers:

- $\exists \rightarrow$ Existential quantifier (there exists)
- $\forall \rightarrow$ Universal quantifier (for all)

Incorrect use of quantifiers can produce invalid results.

Incorrect Expression:

$$\{ t \mid \text{Employee}(t) \text{ AND } \forall d(\text{Department}(d) \rightarrow t.\text{department}=d.\text{department}) \}$$

Error Explanation:

The expression means:

"Find employees who belong to every department."

Example:

Departments:

HR

IT

Finance

An employee cannot belong to all departments simultaneously.

Therefore:

Result = Empty Set

Correct Expression:

$$\{t \mid \text{Employee}(t) \text{ AND} \\ \exists d(\text{Department}(d) \text{ AND} \\ t.\text{department}=d.\text{department})\}$$

Explanation:

The corrected expression means:

"Find employees who belong to at least one valid department."

Debugging Steps:

Step 1:

Identify logical operator.

Step 2:

Check quantifier meaning.

Step 3:

Compare condition with database relationship.

Step 4:

Replace incorrect quantifier.

Result:

The relational calculus expression now returns valid employee records.